Lens 2D Render Engine

By Dheeraj Karki
on -15-Apr-2025

Agenda

- About Lens
- Motivation
- Contributors
- Investigation
- Constrained
- Approach
- Code walk through
- Q&A

About Lens

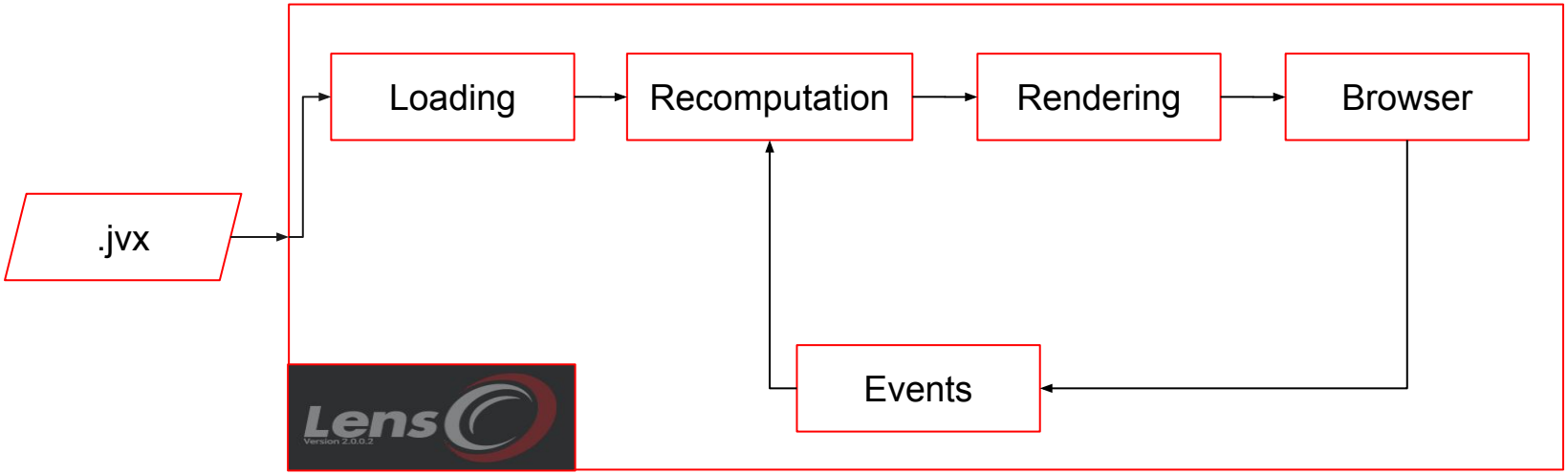
Lens is a web application that can be used both as a standalone product and as an embedded component to render, measure and interact with 2D and 3D designs in a web browser.

Lens is highly configurable and has a significant level of customisation to try to make sure it looks, feels and functions in the same way that a hosting application does.

Lens aims to render designs as close as possible to the way that Impact does, providing a reliable view on a different platform and scale. [Credit: Paul Hyatt]

More Details: https://youtrack.ardensoftware.com/articles/LENS-A-28/Introduction

impact
nServer
WEBcnx
.jvx
Lens
Version 2.0.0.2


.jvx
Loading
Recomputation
Rendering
Browser
Events
Lens
Version 2.0.0.2

Motivation

When faced with heavy drawing loads in the lens, we encountered challenges with responsiveness, particularly in panning and zooming to specific areas. This not only disrupted our workflow but also hindered our ability to efficiently navigate and manipulate the content.

By optimizing the system to handle heavy drawing loads more effectively, we can ensure smoother navigation and better control over the zooming and panning functionalities.

More more details: LENS-135 Rendering Engine Changes

Contributors

Role	Contributors
Proposer	Dheeraj Karki
Consultant	David Fry, Paul Hyat, Carl Manifold and Geoff Birch
Reviewers	Paul Hyat
QA	Geoff Briggs, John Vel and Matthew Hewitt
Stakeholder	Matthew Hewitt
Developer	Dheeraj Karki

Investigation

During the investigation, we identified the following issues responsible for slow rendering:

1. Frequent render state changes.
2. Recomputation of geometry with each event.

Addressing these problems will likely lead to significant improvements in rendering performance.

Constrained

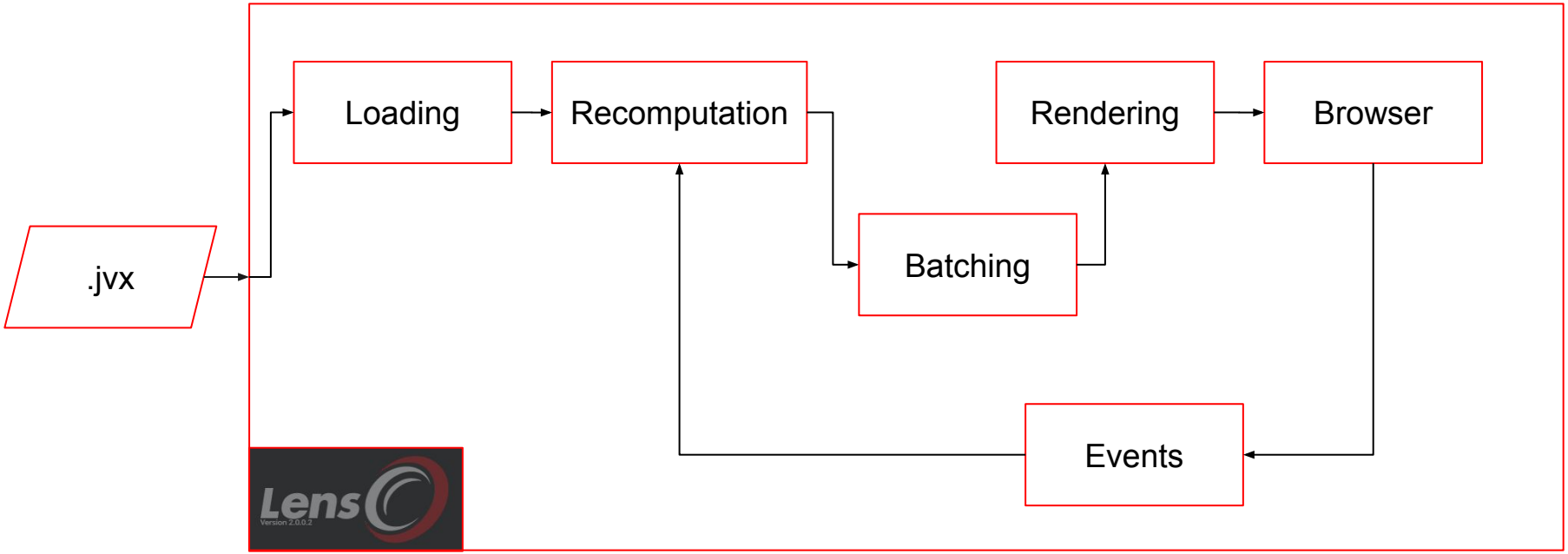
Since this was a new approach and not yet proven, we couldn't adopt it without thorough testing. Therefore, we established some constraints to enable us to proceed with the current implementation while also exploring the new approach:

1. We should avoid or minimize changes to the current renderer.
2. We need to provide statistics demonstrating the efficiency gains of the new approach.
3. The behavior should be configurable to allow for easy switching between the old and new renderers.
4. Changes should be minimal and confined to the new module, with minimal alterations to the entities' draw method.

Approach

Frequent render state changes.

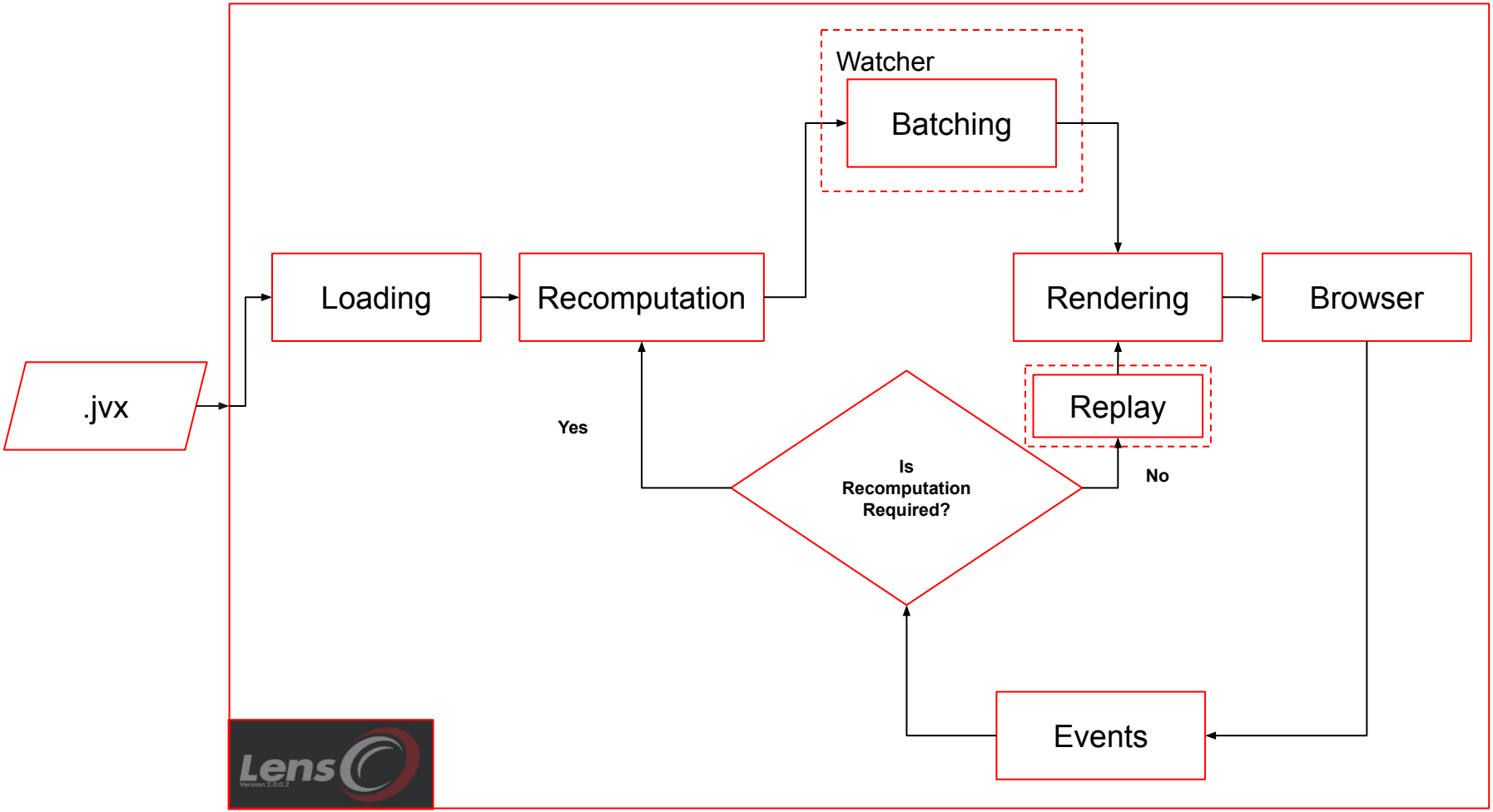
During the investigation, we also discovered that the render state changes frequently, including changes in fill style, line style, and text style. To address this inefficiency, we proposed batching entities with the same style to minimize render state changes, thus improving performance.


.jvx
Loading
Recomputation
Rendering
Browser
Batching
Events
Lens
Version 2.0.0.2

Approach

Recomputation of geometry with each event.

During the investigation, we discovered that we were recomputing the geometry with each event, such as zooming, panning, and mouse interactions. To address this inefficiency, we proposed storing the final geometry and only recomputing it when necessary. Otherwise, we will render the precomputed geometry, reducing unnecessary calculations and improving performance.


Watcher
Batching
Loading
Recomputation
Rendering
Browser
.jvx
Yes
Replay
Is Recomputation Required?
No
Events
Lens
Version 2.0.0.2

Code walk thorugh

Renderer Design

We developed a **BatchedGraphics2D** class as an extension of **Graphics2D**, employing a batching technique to streamline draw calls in an optimized manner. As BatchedGraphics2D extends Graphics2D, there's no need to modify anything in Graphics2D, making it easy to replace **Graphics2D** with **BatchedGraphics2D** when necessary.

This approach draws inspiration from the decorator design pattern with some modification.

Batching Design

The batching approach is divided into multiple subclasses, each assigned specific responsibilities:

- **BatchManager**: Responsible for maintaining a pool of batches.
- **BatchRenderer**: Responsible for rendering primitives on the HTML canvas.
- **BatchContainer**: Contains drawable primitives.
- **Style**: Determines how shapes are drawn.
- **BatchConfig**: Defines how to streamline primitives.
- **Primitive**: Namespace for native drawable objects.
- **LineType**: Determines how lines are drawn.

Watcher Design

We need to monitor and track the behavior of objects whether it's for debugging, performance optimization, or gaining insight into the inner working of an application, developers often require a mechanism to observe and record the activities of specific objects. The concept of a "Watcher" class has emerged as a powerful tool while working on Rendering Engine Changes , offering simple yet effective means of capturing the behavior of the canvas context.

The Watcher is a specialized class that plays the role of a silent observer, diligently recording the activity of target objects belonging to a certain class. By recording the activity of the object we can analyze and optimize the code effectively.

Watcher Design

At its core, the Watcher class consists of a constructor that takes two arguments: **target** and **exclusions**. The target the argument is required and represents the object whose activities the Watcher will monitor. On the other hand, exclusions is an optional argument that allows developers to specify functions they don't want the Watcher to record. By carefully selecting these exclusions, developers can fine-tune the Watcher to focus on the most critical aspects of the target object's behavior.

More Detail: [The Watcher: A Silent Sentinel Recording Object Class Activity | Knowledge Base (ardensoftware.com)]

Watcher Design

We created specialized watcher which is responsible for watching a particular object like in our case it is 2d context.

So, **WatchedContext** is responsible to record any canvas' context object class activity.

Q&A

Thanks